FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Larrion Luces
Section:	
Professor:	
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

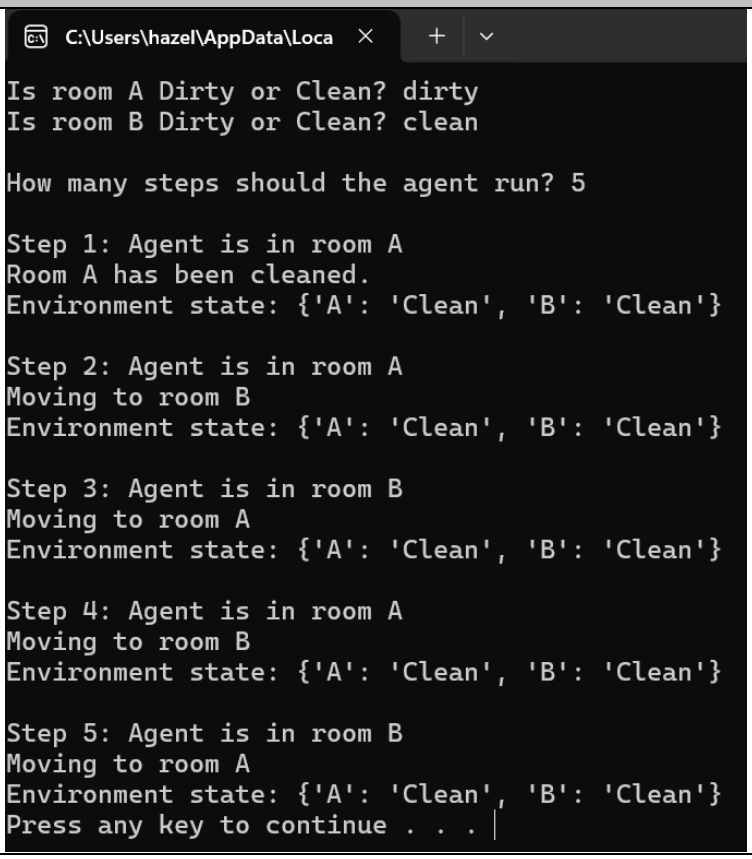
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

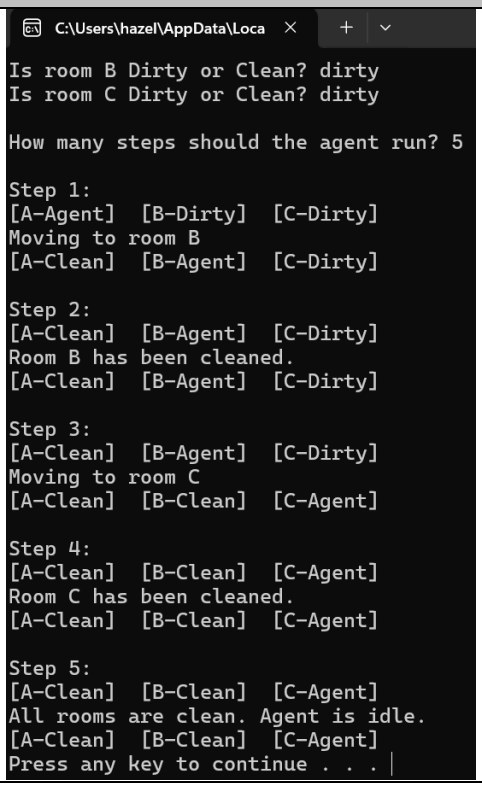
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
import random

def get_room_state(room_name):
    """Ask the user for a valid initial room state."""
    while True:
        state = input(
            f"Enter the initial state of Room {room_name} "
            "(Dirty/Clean): "
        ).strip().capitalize()
```

```
        if state in ["Dirty", "Clean"]:
            return state

        print("Invalid input. Please enter Dirty or Clean.")

def is_dirty(environment, room):
    """Return True when the selected room is dirty."""
    return environment[room] == "Dirty"

def clean_room(environment, room):
    """Change a dirty room's state to clean."""
    environment[room] = "Clean"

def display_grid(environment, agent_location):
    """Display the rooms and the agent's current location."""
    room_display = []

    for room, state in environment.items():
        if room == agent_location:
            room_display.append(f"[{room}: {state} | AGENT]")
        else:
            room_display.append(f"[{room}: {state}]")

    print(" -- ".join(room_display))

class RuleBasedVacuumAgent:

    def __init__(self, starting_location="A"):
        self.location = starting_location

    def perceive_and_act(self, environment):
        """
        Perceive the current room and select an action.

        Rule 1: Clean the current room if it is dirty.
        Rule 2: If the current room is clean, randomly select
                another dirty room.
        Rule 3: Stop if every room is already clean.
        """

        if is_dirty(environment, self.location):
            clean_room(environment, self.location)
            return f"Cleaned Room {self.location}"
```

```
        dirty_rooms = [
            room
            for room, state in environment.items()
            if state == "Dirty" and room != self.location
        ]

        if dirty_rooms:
            target_room = random.choice(dirty_rooms)
            old_location = self.location
            self.move(target_room)

            return (
                f"Moved from Room {old_location} "
                f"to dirty Room {target_room}"
            )

        return "No action—all rooms are clean"

    def move(self, target_room):
        """Move the agent to the selected room."""
        self.location = target_room

print("RULE-BASED VACUUM CLEANER AGENT")
print("Bonus version with Rooms A, B, and C")
print()

environment = {
    "A": get_room_state("A"),
    "B": get_room_state("B"),
    "C": get_room_state("C")
}

while True:
    starting_location = input(
        "Enter the agent's starting room (A/B/C): "
    ).strip().upper()

    if starting_location in environment:
        break

    print("Invalid room. Please enter A, B, or C.")

while True:
```

```
    try:
        number_of_steps = int(
            input("Enter the number of simulation steps: ")
        )

        if number_of_steps > 0:
            break

        print("The number of steps must be greater than zero.")

    except ValueError:
        print("Please enter a valid whole number.")

agent = RuleBasedVacuumAgent(starting_location)

print("\n=== INITIAL ENVIRONMENT ===")
display_grid(environment, agent.location)

for step in range(1, number_of_steps + 1):
    print(f"\n=== STEP {step} ===")

    print(f"Perception: Room {agent.location} is "
          f"{environment[agent.location]}")

    action = agent.perceive_and_act(environment)

    print(f"Action: {action}")
    print(f"Current location: Room {agent.location}")
    print("Current environment:")
    display_grid(environment, agent.location)

    if all(state == "Clean" for state in environment.values()):
        print("\nAll rooms are clean. The simulation is complete.")
        break

else:
    print("\nThe requested number of steps has been completed.")

print("\n=== FINAL RESULT ===")
print(f"Agent's final location: Room {agent.location}")
display_grid(environment, agent.location)
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. **The agent cleans its current room if it is dirty. If the current room is clean, it looks for another dirty room and moves there. In the bonus version, it randomly selects a destination where multiple dirty rooms are available are available. It stops when every room is clean.**
2. **The agent perceives the state of its current room. It cleans when that room is dirty. If the room is already clean, it searches the environment for dirty rooms and moves to one of them.**
3. **The agent determines that no cleaning is necessary. The simulation displays that all rooms are clean and stop without changing the environment.**
4. **The rooms can be stored in a dictionary instead of writing separate rules for Rooms A and B. The agent can search the dictionary for dirty rooms and select one as it next destination. This approach allows additional rooms to be added without rewriting the cleaning rule.**